

System Requirements

Operating System

- Linux only. *run Linux via WSL 2 or use a virtual machine.*
- Recommended distro: Ubuntu 20.04+ or Debian-based.
- Kernel version: $\geq 5.x$ with eBPF support enabled.

Hardware

- x86_64 CPU with performance counters (Intel/AMD).
- This is Optional: NVIDIA GPU with CUDA Toolkit (for GPU profiling).

Permissions

- Root access or sudo.
 - Kernel parameter: `perf_event_paranoid = 1` or lower.
 - Enable memory locking: `ulimit -l unlimited`
- Strobelight supports dozens of profiler “types” – e.g. CPU cycle sampling, memory allocation tracking, off-CPU (idle/block time) profiling, language-specific profilers for Java/Python, and even GPU kernel tracing. Each profiler type focuses on a different aspect of performance, but Strobelight unifies their configuration and data collection, so you can easily gather comprehensive performance traces from complex systems.

Using for java:

the JVM normally doesn't preserve frame pointers in JIT-compiled code by default (which makes native stack sampling difficult). To fix this, run Java application with “-XX:+PreserveFramePointer”.

This JVM flag ensures that JIT compiled methods keep their frame pointers, allowing Strobelight's stack unwinding to capture full Java call stacks

For **CPU usage (on-CPU time)**, you would use a CPU sampling profiler (often configured to sample CPU cycles or timer interrupts). capture which methods are consuming CPU and how much.

For **off-CPU time (idle or waiting)**, Strobelight has an “offcpu” profiler that tracks when threads are blocked (e.g., waiting on I/O or locks).

For **memory allocations**, there is a profiler powered by jemalloc that can track allocations (though for Java, which uses its own heap, this might not directly capture Java object allocations unless JVMTI agents are used – but native allocations via JNI can be profiled).

For **GPU usage**, the **gpuevent** profiler can trace CUDA kernel launches. If your Java AI uses GPU via libraries (like TensorFlow Java or DeepLearning4J with CUDA backend), you might enable this to see GPU kernel activity initiated by the Java process

For **language-specific profiling**, Strobelight's event-based profilers for Java can capture Java method stacks. Internally, this might work by sampling threads within the JVM via BPF uprobes or perf events that know how to traverse Java frames.

You can run multiple profiling sessions focusing on different aspects: CPU vs off-CPU, or profile different phases of the workload (startup vs steady-state). You could also set up **continuous profiling**: for example, configure Strobelight to sample your Java service every N minutes automatically. At Meta, engineers often set continuous profiles via config so that data is always available when needed

Dependencies:

CMake/Make for building
Clang/LLVM for eBPF
Libbpf (BPF library)
CUDA toolkit (for gpu profilers)
JDK that supports -XX:+PreserveFramePointer for method level specificity

Kernel Config:

CONFIG_BPF is enabled
If using flame graphs from kernel stacks, CONFIG_UNWINDER_ORC enabled
Permissions -- Root access

If errors about: RLIMIT_MEMLOCK -- check ulimit -1

For Data management:

eBPF-- extended Berkeley Packet Filter

Attaches to the linux kernel, small programs that can observe filter and act on things like:

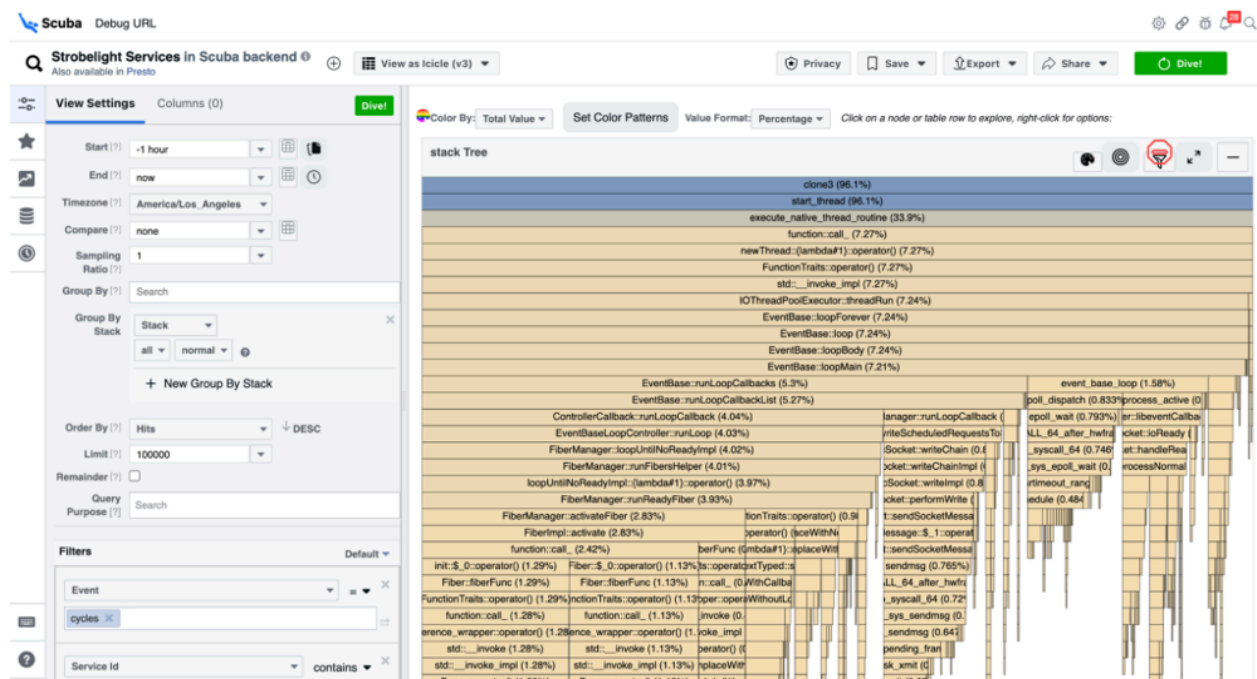
- Sys calls
- Scheduler activity
- Hardware performance counters

Strobelight samples hardware performance counters (cpu cycles, power states), we can then use eBPF, employing the tools like perf_event or PMU counters. We can map these to specific methods by tracing the stack

Strobelight's output:

After running a profile, Strobelight yields a lot of data: typically stack traces with associated CPU cycles count or event counts.

FLAME GRAPH and the Scuba UI



- each box represents a function in the stack trace, and wider boxes mean more time (samples) spent in that function
- If an AI job is not fully utilizing the CPU, an off-CPU profile can reveal bottlenecks like synchronization -- refactoring through optimizing concurrency

SCUBA - Scalable Universal Cross-Bar for Analytics

- Unfortunately this UI isn't open source -- we would have to output the necessary Strobelight data to a file and format it properly so that we can make a flame graph, though that type of representation isn't wholly necessary. [This tool might work](#)

Running a Strobelight Profiling session

- Start java application
- Identify the PID for the process, via jps
- To run the profiler targeting the java process:

```
$ sudo strobelight/src/_build/profilers/cycles_profiler -p <Java_PID> -d 60 -o java_profile.fbdata
```

- Strobelight will load the appropriate eBPF program into the kernel to begin sampling

Jordan Rome – “*Strobelight: A profiling service built on open source technology,*” Engineering at Meta (Jan 2025)
engineering.fb.com

Meta Engineering Blog – *Strobelight combines open source profilers (CPU, memory, Python, Java, GPU, etc.) into one service*
engineering.fb.com

Meta AtScale Conference – “*Strobelight is a daemon on all fleet hosts... composed of multiple subprofilers for CPU, memory, Python, etc., triggered on events or schedule,*” (AI Observability talk, 2023)
atscaleconference.com

Strobelight Open Source Repository – *README and Usage examples*
github.com

Hacker News (comment by Meta engineer) – *Importance of frame pointers in all user-space binaries for stack walking*

news.ycombinator.com

LinkedIn – “*Strobelight: New Era of GPU Performance Observability*” (Olu O.), describing Strobelight GPU profiling and visualization in Scuba

[linkedin.com](https://www.linkedin.com)

Adam Barth (Meta) – *Spark AI Summit talk (2020): Using Production Profiles to Guide Optimizations* – notes on using Strobelight (perf-based, minimal Java context) vs async-profiler (Java-specific)

[slidestalk.com](https://www.slidestalk.com)

Polar Signals Blog – *Discussion on eBPF profiling for Python/JVM with 19Hz sampling and <1% overhead*

news.ycombinator.com

AtScale AI Observability – *Use of BPF for GPU events (gpusnoop) and safe, low-overhead profiling*